MultiSportPredict
Czech Liga Pro Match & Command Guide

Updated 10 October 2026 • Windows / Git Bash / VS Code terminal
Use the copied-page workflow to run an upcoming match without an API token. Read the saved report in your terminal, push its Discord card, then collect final results and grade. This guide covers Czech Liga Pro only; ordinary tennis commands are in the final appendix.

DATA INGESTION: QUICK START

Open Git Bash (or select Git Bash in VS Code → Terminal → New Terminal). Paste this complete block to collect Czech Liga Pro finished-match history without a paid token:

```
cd /c/MultiSportPredict
source venv/Scripts/activate
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 10
--auto-train
venv/Scripts/python.exe -u ingest_table_tennis.py --stats
```

This merges verified final scores, exports history and attempts training. It does not fetch MyStake prices; use the copied-page match workflow below for current prices. A small history is collected data, not a trained recommendation.
Keep collecting every 30 minutes (leave this terminal open; Ctrl+C stops):

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100
--watch --interval 1800 --auto-train
```

Export the existing store to CSV without collecting another page:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --export-only
```

Ingestion creates no prediction card to push. After running a match, the Discord command is:
venv/Scripts/python.exe -u run_table_tennis.py --push latest
All other sports:
https://docs.google.com/document/d/1W21N_tqpREVkbQhdGND1uclTc7YvfvsYQRLcC__ZopI/edit

Action index

- DATA INGESTION: QUICK START
- Start Git Bash or VS Code
- Run an upcoming match: complete free workflow
- Run a second match or refresh prices
- View saved results in your terminal
- Preview and push to Discord
- Read probabilities, prices and EV

- DATA INGESTION: collect and refresh table-tennis history
- Train and inspect player history
- Grade predictions after matches finish
- Manually update a verified MyStake final result
- Import a verified JSON export or source snapshot
- Check win percentage versus point-cover percentage
- API INGESTION: setup, backfill and fixture refresh
- Troubleshooting and dependency repair
- File locations and command lookup
- Ordinary tennis: ATP / WTA appendix

Start Git Bash or VS Code

Windows Start menu: search Git Bash and open it. In VS Code: Terminal → New Terminal → choose Git Bash from the terminal profile dropdown. Paste commands into that terminal; opening a .py file does not execute it.

```
cd /c/MultiSportPredict
source venv/Scripts/activate
pwd
venv/Scripts/python.exe --version
```

Expected: /c/MultiSportPredict, a Python version and usually (venv). All commands use the explicit project interpreter. -u displays output immediately. If a collector occupies this terminal, press Ctrl+C once and wait for the $ prompt before running a match, or use a separate Git Bash terminal for the match.

Run an upcoming match: complete free workflow

Copy and save the sportsbook text

Open the individual MyStake match, not the list of fixtures. Confirm Czech Liga Pro, the two players, the start time and best-of-five format. Expand the markets you want. Copy visible text containing Table Tennis, Czechia, Czech Liga Pro, both player names, date/time, market labels and both prices. Click inside the loaded match panel, press Ctrl+A and Ctrl+C. Check the copied text contains those items; screenshots cannot be parsed as page text.

```
notepad.exe 'C:\MultiSportPredict\data\table_tennis\current_match.txt'
```

Paste into Notepad and press Ctrl+S. In VS Code the same file is at MultiSportPredict → data → table_tennis → current_match.txt. It is a text file. Replace its contents with one match page at a time. Joined prices such as 11.41 and Over (73.5)1.76 are supported. Do not manually add a league heading that was absent from the actual page.

Run the match and print the report

Run this entire block. Enter the actual names, the exact MyStake prematch match URL, and the confirmed start with an explicit timezone offset. Run within five minutes of copying the fresh page and before the match starts.

```
read -r -p "Match (First Player vs Second Player): " TT_MATCH
read -r -p "MyStake prematch match URL: " TT_URL
read -r -p "Confirmed start (YYYY-MM-DDTHH:MM:SS with timezone offset): " TT_START
venv/Scripts/python.exe -u run_table_tennis.py --match "$TT_MATCH" --mystake-url
"$TT_URL" --mystake-text-file data/table_tennis/current_match.txt --start-utc
"$TT_START" --best-of 5
```

For Eastern time on 10 October 2026 the offset is -04:00; standard time uses -05:00. Confirm the actual date/offset for your fixture. --best-of 5 confirms the real format. A successful run prints win probabilities, expected points/games, all recognized priced markets with EV, reference projections, observations, a saved report path and a tt_... run ID. Save that ID.

Optional automatic browser attempt

This attempts to read the page without the copied file. MyStake browser capture has been unreliable in this project; use the copied-page workflow if it fails.

```
venv/Scripts/python.exe -u run_table_tennis.py --match "$TT_MATCH" --mystake-url
"$TT_URL" --start-utc "$TT_START" --best-of 5
```

Run a second match or refresh prices

For another match, replace current_match.txt with its freshly copied full page, save, and rerun the complete prompt-and-run block above with that match's names, URL and start. Each successful run creates a separate saved report. Do not combine two pages in one text file. To update prices for the same upcoming match, recopy the live page and rerun; a new run ID preserves the new capture. Resaving old text does not update sportsbook prices. The current MyStake workflow is prematch only; it cannot use stale prematch text as a live forecast.

View saved results in your terminal

Show the most recent report, or list the last ten saved runs:

```
venv/Scripts/python.exe -u run_table_tennis.py --show latest
venv/Scripts/python.exe -u run_table_tennis.py --list-runs
```

Show a particular saved report:

```
read -r -p "Saved run ID (tt_...): " TT_RUN
venv/Scripts/python.exe -u run_table_tennis.py --show "$TT_RUN"
```

Already saved examples — these display the original captured predictions and prices:

```
venv/Scripts/python.exe -u run_table_tennis.py --show
tt_d607a0e572314dc89db15adc0189a350
venv/Scripts/python.exe -u run_table_tennis.py --show
tt_ae4708ed150641bbab07de865a265979
```

The first is Vaclavik–Havrda; the second is Holovatiuk–Briza. --show reads a saved report without collecting prices or creating another forecast.

Preview and push to Discord

Use the run ID printed by your match run. A match/event ID and a tt_... saved run ID are different.

```
read -r -p "Saved run ID to send (tt_...): " TT_RUN
venv/Scripts/python.exe -u run_table_tennis.py --push "$TT_RUN" --dry-run
```

The dry run previews the payload without sending. To send the report:

```
venv/Scripts/python.exe -u run_table_tennis.py --push "$TT_RUN"
```

Push uses configured Discord destinations and the latest formatter with the report's original prices. It may update the prior tracked card for that fixture. Model Notes is removed from Discord; the research/price basis remains in the short summary. Collection, training and grading do not generate new prediction cards. After those actions, the push command above sends an associated saved prediction; it does not create a grading-results card.

Read probabilities, prices and EV

Priced markets: the copied page or API supplies actual paired sportsbook odds. The report shows decimal odds, model win probability and EV. EV = P(win) × (decimal odds − 1) − P(loss). Pushes return the stake and contribute zero. +5% EV means a calculated 0.05 units of expected profit per unit staked under the model, not a guarantee.

Model references: the model generates probabilities at reference lines. A sportsbook quote may be absent for those lines. Without actual odds there is no sportsbook price or EV to report. A quote for Over 73.5 cannot price Over 77.5. If your copied page supplies a recognized market and both actual prices, it appears in the priced section. Missing or ambiguous lines remain projections.

Supported market families: match winner; match total games; game handicap; match total points; point handicap; first game winner; first game total points; first game point handicap. Actual availability depends on the source. In handicaps, A is the first player displayed by the source; the displayed B handicap has the opposite sign. Current market-anchored output does not provide an independently trained betting edge.

DATA INGESTION: collect and refresh table-tennis history

Collect visible recent finished Czech Liga Pro results once, export CSV and attempt training:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 10
--auto-train
```

Keep collecting with a 30-minute delay after each cycle:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100
--watch --interval 1800 --auto-train
```

Leave that terminal open; Ctrl+C stops. Discovery, each accepted/rejected match and the waiting interval are printed. Recent results are limited, not a complete backfill. Access can fail; only complete verified legal scores are saved. Duplicate results do not create extra events. Check store counts, export CSV or inspect files:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --stats
venv/Scripts/python.exe -u ingest_table_tennis_public.py --export-only
explorer.exe 'C:\MultiSportPredict\data\table_tennis'
```

Collect one known completed public match:

```
read -r -p "Completed Sofascore table-tennis match URL: " TT_RESULTS_URL
venv/Scripts/python.exe -u ingest_table_tennis_public.py --match-url "$TT_RESULTS_URL"
--auto-train
```

Sofascore history uses different player/event IDs from MyStake. Automatic reconciliation is still unresolved. Collecting a result under a Sofascore ID alone does not grade a MyStake prediction.

Train and inspect player history

```
venv/Scripts/python.exe -u train_table_tennis.py
venv/Scripts/python.exe -u train_table_tennis.py --export
data/table_tennis/training_rows.json
venv/Scripts/python.exe -u train_table_tennis.py --diagnostic
```

The commands respectively fit the model, export eligible feature rows, and run the losing-point Poisson diagnostic. Training requires at least 150 eligible matches; each eligible match needs eight prior matches for each player. An insufficient-data message means no model was fitted. Completed results alone are not equivalent to a validated betting model. The diagnostic is not a separate recommendation.

Grade predictions after matches finish

Refresh completed history using your chosen source, then grade:

```
venv/Scripts/python.exe -u ingest_table_tennis_public.py --recent --max-events 100
venv/Scripts/python.exe -u grade_table_tennis.py
```

Grading needs the saved report’s exact event/player identities, finished status, and all legal per-game point scores ending at three game wins. A final 3–1 match score alone is insufficient. For MyStake reports, use the verified manual-result update below when automatic identity reconciliation is unavailable.

Output reports graded selections, unfinished/invalid runs, hypothetical one-unit profit and prospective probability metrics where available. These are research outcomes; the engine did not record that you placed those bets. Regrading updates records for corrected results. Discord push continues to send the saved prediction, not this grading summary.

Manually update a verified MyStake final result

Use this only after verifying the final result page is the same fixture: both full names, date/time and every game's points. Open the saved report first. Enter game point pairs in source A/B order, not winner-first order. TT_FINAL_SCORES must be a JSON array of actual two-number game-score arrays. Use all completed games and the real result source URL. No scores are prefilled.

```
read -r -p "Saved run ID for the completed match (tt_...): " TT_RUN
venv/Scripts/python.exe -u run_table_tennis.py --show "$TT_RUN"
read -r -p "Verified game point pairs in A/B order, JSON array: " TT_FINAL_SCORES
read -r -p "Verified final-result page URL: " TT_RESULT_URL
```

Prepare a validated import file. This command preserves the stored event/player identities; it does not guess cross-provider mappings. The observation time is now; no match end time is invented.

```
venv/Scripts/python.exe -c 'import sys,json; from pathlib import Path; from
core.historical_storage import TableTennisStore; from table_tennis.contracts import
utcnow,validate_event; r=TableTennisStore().run(sys.argv[1]); e=dict(r["event"]);
e.update(status="finished",game_scores=json.loads(sys.argv[2]),current_points=[0,0],re
trieved_utc=utcnow(),source_url=sys.argv[3]); e.pop("end_utc",None);
e=validate_event(e); p=Path("data/table_tennis/verified_final_result.json");
p.write_text(json.dumps({"events":[e],"quotes":[]},indent=2),encoding="utf-8");
print("Validated result written:",p)' "$TT_RUN" "$TT_FINAL_SCORES" "$TT_RESULT_URL"
```

Only after that command succeeds, import and grade:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --import-file
data/table_tennis/verified_final_result.json
venv/Scripts/python.exe -u grade_table_tennis.py
venv/Scripts/python.exe -u train_table_tennis.py
```

If validation fails, correct the actual input before importing. Unknown or incomplete final points must remain ungraded. This manually confirmed update does not claim that the engine independently verified the result-page identity. Importing a corrected result uses the same workflow and retains the database's event versions.

Import a verified JSON export or source snapshot

A normalized engine export has events/quotes arrays. Raw DOM snapshots, page-text files and CSV exports use different formats.

```
read -r -p "Verified normalized events/quotes JSON path: " TT_IMPORT
venv/Scripts/python.exe -u ingest_table_tennis.py --import-file "$TT_IMPORT"
```

A raw Sofascore DOM capture produced by the public collector uses this importer:

```
read -r -p "Verified public DOM snapshot JSON path: " TT_SNAPSHOT
venv/Scripts/python.exe -u ingest_table_tennis_public.py --snapshot-file
"$TT_SNAPSHOT"
```

After normalized results are imported, use the grading/training commands above. Exported results.csv is an inspection file, not a normalized JSON importer input.

Check win percentage versus point-cover percentage

Use an exact stored player ID, a real signed point handicap and one bookmaker. The analysis compares match wins and covers on the same historical priced matches; it needs matching pre-match quotes and settled results.

```
read -r -p "Exact stored player ID: " TT_PLAYER_ID
read -r -p "Signed player point handicap: " TT_HANDICAP
read -r -p "Bookmaker: " TT_BOOK
venv/Scripts/python.exe -u run_table_tennis.py --win-cover-player-id "$TT_PLAYER_ID" --point-handicap "$TT_HANDICAP" --bookmaker "$TT_BOOK"
```

Player IDs appear in results.csv and saved report JSON. Insufficient matching samples remain unavailable. This comparison is descriptive, not EV.

API INGESTION: setup, backfill and fixture refresh

Optional: obtain a separate BetsAPI token with verified Czech Liga Pro completed game-point coverage and the required odds entitlement. Your existing The Odds API key has no table-tennis coverage, and a Bet365 login is not an API token. Enter tokens privately in your own terminal.

```
winpty venv/Scripts/python.exe configure_table_tennis.py
venv/Scripts/python.exe -u configure_table_tennis.py --check
venv/Scripts/python.exe -u ingest_table_tennis.py --provider betsapi --days 90 --no-quotes --auto-train
```

Refresh current fixtures and available sportsbook quotes (requires a BetsAPI token and odds coverage):

```
venv/Scripts/python.exe -u ingest_table_tennis.py --provider betsapi --days 0
```

With provider access, run by names:

```
read -r -p "Match (First Player vs Second Player): " TT_MATCH
venv/Scripts/python.exe -u run_table_tennis.py --provider betsapi --match "$TT_MATCH"
```

Refresh recent provider results and grade provider-matched runs:

```
venv/Scripts/python.exe -u ingest_table_tennis.py --provider betsapi --days 1 --no-quotes --auto-train
venv/Scripts/python.exe -u grade_table_tennis.py
```

Use --push with that report's tt_... ID. Provider history/quotes do not automatically reconcile earlier MyStake identities.

Troubleshooting and dependency repair

No $ prompt: an action still occupies the terminal. Watch status says when it is waiting; Ctrl+C stops it. A lone > prompt means an unfinished quote/multiline command; Ctrl+C, then paste a complete command. File not found: check pwd and cd /c/MultiSportPredict.

Copied page rejected: ensure the actual page contains all league labels, both players and paired market prices. Recopy current prices within five minutes. Match start passed: use --show for the saved report; do not change the start to a fake future time. Browser source blocked: use fresh copied text for MyStake; public results retain old data on failure.
No EV: actual prices are absent for that market/line. No trained model: collect eligible history and run training. Zero grades: final scores or exact event identities are missing. CSV access denied: close results.csv in Excel before another export.
Dependency repair only when the error says dependencies/browser are missing:

```
venv/Scripts/python.exe -m pip install -r requirements-table-tennis.txt
venv/Scripts/python.exe -m playwright install chromium
```

File locations and command lookup

Project: C:\MultiSportPredict
Copied match: data\table_tennis\current_match.txt
Canonical history: multisport_history.db
CSV export: data\table_tennis\results.csv
Raw completed captures: data\table_tennis\public_raw
Fitted model: data\table_tennis\model.json (only after training succeeds)
Saved reports: output\table_tennis\tt_....json and tt_....md
Command guide local copy: TABLE_TENNIS_COMMANDS.md

```
venv/Scripts/python.exe run_table_tennis.py --help
venv/Scripts/python.exe ingest_table_tennis_public.py --help
venv/Scripts/python.exe ingest_table_tennis.py --help
venv/Scripts/python.exe train_table_tennis.py --help
venv/Scripts/python.exe grade_table_tennis.py --help
```

Ordinary tennis: ATP / WTA appendix

Use run_tennis.py or run_tennis_full.py for ordinary tennis. These have different storage/review conventions and Discord flags from run_table_tennis.py.

```
venv/Scripts/python.exe -u ingest_tennis.py
```

Standard match run: enter the actual fixture properties rather than accepting a default tournament.

```
read -r -p "Match (First Player vs Second Player): " TEN_MATCH
read -r -p "Surface (hard/clay/grass/carpet): " TEN_SURFACE
read -r -p "Tournament: " TEN_TOURNAMENT
read -r -p "Round: " TEN_ROUND
read -r -p "Best of (3/5): " TEN_BEST_OF
read -r -p "Actual match date (YYYY-MM-DD): " TEN_DATE
venv/Scripts/python.exe -u run_tennis.py --match "$TEN_MATCH" --surface "$TEN_SURFACE"
--tournament "$TEN_TOURNAMENT" --round "$TEN_ROUND" --best-of "$TEN_BEST_OF"
--game-date "$TEN_DATE"
```

Push a selected review row from the standard runner:

```
read -r -p "Review row number: " TEN_ROW
```

```
venv/Scripts/python.exe -u run_tennis.py --push "$TEN_ROW"
```

The full research-package runner fetches a source fixture/profile/odds package where supported; source access and identifiable history remain necessary:

```
venv/Scripts/python.exe -u run_tennis_full.py --match "$TEN_MATCH" --refresh
venv/Scripts/python.exe -u run_tennis_full.py --push
```

Grade logged ordinary-tennis predictions with the ordinary project grader. The full research-package runner saves a research report and does not automatically register a graded bet.

```
venv/Scripts/python.exe -u grade_predictions.py --pending --sport tennis
venv/Scripts/python.exe -u grade_predictions.py --auto --sport tennis --report
```

Push the ordinary-tennis grading record to Discord:

```
venv/Scripts/python.exe -u grade_predictions.py --report --sport tennis --push-discord
```